Scanning & hardening authentication CVEs in ARM-based IoT devices

Adrian Ampuero
Florida International University,
College of Computing and Engineering
Miami, FL USA

Lindsey Blais
Florida International University,
College of Computing and Engineering
Miami, FL USA

Jessica Perez Ferrer
Florida International University,
College of Computing and Engineering
Miami, FL USA

***Abstract*—This paper examines the current state of IoT security and describes a standardized framework for vulnerability management and remediation. Specifically, many IoT devices run on ARM-based processors, and are subject to separate vulnerabilities not found in Windows or Linux devices due to the CPU architecture. The vast majority of mobile electronics, approximately 75 billion devices, whether they are smartwatches, touch displays, cameras, wireless routers, or embedded computing devices, run on ARM-based processors. Discovering and patching vulnerabilities on these devices can be a daunting task for enterprise administrators. In this study, we demonstrate the OWASP Firmware Security Testing Methodology to scan an ARM-based IoT device. Using traditional vulnerability scanners to find authentication-based Common Vulnerability & Exposures (CVE), we then compare these results to the firmware scan and analysis from the EMBA, an embedded analyzer. We then explore how improved implementations can be designed for previously proposed binary patching tools. The goal of this project is to demonstrate a workflow for cost-effective vulnerability management without requiring vendor support or source code access and propose a complete independent vulnerability lifecycle solution for ARM-based IoT devices.**

Keywords—IoT, ARM processor, EMBA, Vulnerability Management, Authentication, OWASP, FSTM, binary patching

I. Introduction

The widespread surge in the use of IoT (Internet of Things) devices has wildly transformed modern day computing into a vast array of ways. IoT devices (i.e cameras, wireless routers, smartwatches, drones) more colloquially known as "smart devices", allow for seamless connectivity that can automate and alleviate everyday tasks. Devices that rely on this hardware and software possess components such as processors, sensors, actuators, and various connectivity components that interact with networks to process and relay data. The technology that is embedded into these devices plays a crucial role in processing information and delivering it where it is intended to go, expectedly in a secure manner. However, while the increasingly growing use of IoT devices offers convenience and ease of use, security concerns scale alongside.

Historically, when IoT devices were first introduced, security design was less of a focus as these devices were mainly connected through isolated environments, and a push for mass market limited the design & development stages to short windows. Now, the majority of IoT devices are connected to networks and the internet, exposing them to a vast number of threats.

New technology will always possess a vulnerability at some point in its lifespan. A device that is connected to the internet can be at risk of facing a malicious attack from someplace else on the internet. Some common ways that these devices can be attacked are through denial-of-service, spoofing, jamming, eavesdropping, data manipulation, and man-in-the-middle attacks [1]. The most common issues that IoT devices face are unpatched firmware, weak encryption, vulnerable authentication configurations, and even physical threats such as tampering due to the nature of these devices [22].

With these threats in mind, authentication remains one of the biggest weaknesses within IoT devices. The main goal in authentication is to make sure that the identity of a device is what it claims to be before connecting to a network. When authentication is insecure, like in systems that rely on certain hardware-based authentication or systems that contain unpatched firmware, these systems are left vulnerable to attacks [20].

IoT networks are typically of high complexity in scale, and this complexity can increase over time. Yet their benefit to large enterprise productivity and growth makes IoT devices a foreseen demand because they bring massive convenience and functionality to an organization. Even though IoT is widely adopted globally, only a quarter of IoT initiatives grow beyond the proof-of concept phase due to lack of expertise and high technical complexity. A driving reason for these failures is the high degree of device lifecycle management, because IoT networks typically span geographies. Device management is a critical process to maintain the health of IoT devices and ensure they are communicating over the network. Because of their distribution on the network, the provisioning, authentication, software updates, device configuration, monitored diagnostics, and telemetry data present a unique challenge for enterprise administrators. Authentication plays a vital role in the IoT solutions' ability to sustain a reliable, trustworthy, and integrated environment. [28][29][30]. Owed to the vast quantity of enterprise-level IoT devices that require maintenance, IoT solutions repeatedly fail in sustaining an adequate framework to properly secure these devices. For this reason, the Open Worldwide Application Security Project (OWASP) created the Internet of Things Project to provide guidance to manufacturers, developers, and the global community, as well as aid in understanding the cybersecurity threat landscape within the Internet of Things [31]. The OWASP IoT Security Testing Guide (ISTG) is a comprehensive methodology handbook that can be used to perform penetration tests on many different features of IoT devices such as the memory, physical interface, and user interface [15]. Its purpose is to introduce common terminology among manufacturers, developers, and security researchers to facilitate an understanding amongst these interested parties and maintain adequate communication that

spans decades of effort to improve the security of IoT devices. The operating system of IoT devices normally functions above the embedded software on the integrated circuit boards of these devices, and technologists refer to this embedded software as the "firmware". Firmware is the foundational programming that controls the functionality and hardware resources of modern IoT devices [2][32][35]. Firmware binaries can be any operating system but are typically Linux-based and rarely run on Windows, however this is changing with Microsoft's nascent efforts to produce the Azure Well-Architected Framework for IoT and bring these solutions to the cloud with Windows for IoT. According to Microsoft's Azure best practices for IoT, a good IoT architecture is built upon Reliability to provide continued functionality and forming resilience [33]. Therefore, any interruption or violation of this principle, such as devices that fail to authenticate and gain access to corporate resources, provides an imperative for organizations to design devices for resilience. In support of these efforts, OWASP developed a specific use-case methodology to complement the OWASP ISTG and provide a process for conducting firmware security assessments for IoT devices. The OWASP Firmware Security Testing Methodology (FSTM), a nine-staged security assessment, strengthens the "center of controlling an embedded device"[32] by identifying vulnerabilities and designing strategies to mitigate or remediate them.

In this project, we want to assess the current state of vulnerability analysis in ARM-based devices. ARM refers to a distinct CPU architecture that was pioneered by Acorn Computers Ltd. in 1990 and later became known as Arm Holdings to mass produce the Arm architecture CPUs, of which 99% of all smart or IoT devices run on Arm processors [4][5][8]. Under a joint venture between Apple and VLSI, a company that designed custom integrated circuits (IC), the Advanced RISC Machines (ARM) CPU architecture was designed. Acorn Computers were the original developers of the RISC OS [4], which was an operating system that was designed to run on ARM computers. RISC refers to the Reduced Instruction Set Computer model, which was one of the leading paradigms in computing during the rise of personal computers in the 1980's & 1990's. Specifically, RISC represents the concept that a computer executes one instruction per CPU duty cycle [6]. The RISC model performs well by leveraging a concept called "pipelining" [25], which is a form of parallel processing, where instructions are sent to separate CPU cores for processing, or simply "multiple execution units". This process works in tandem with random access memory (RAM), the disk, and I/O components to schedule the instructions sent to the CPU, where specific ICs are designed and manufactured to process several instructions at once using cache memory [6]. The RISC model differs from the CISC, or Complex Instruction Set Computer, in that only a smaller set of instructions in the ARM architecture are streamlined and executed every single computer cycle, leading to more energy efficient computing [7][13][41][42]. Typically, RISC architecture CPUs are purposed for embedded applications that require performing a specific task or feature, very fast and energy efficient, such as running a laser printer and not necessarily executing user or data applications, thereby offering a significant level of flexibility [6][34]. Using the RISC model, Arm transformed the computer industry by defining the standard for balancing performance and efficiency, making it possible to fit compute power to the smallest devices, yet scale to the largest of systems [41][42]. Their market share of Arm CPUs literally outnumbers the entire human population by 40-fold, 325 billion CPUs! A landmark year for Arm, was in 2010 when they released the Armv8 architecture [7][13] helping the industry progress into new computing domains in IoT, automotive, & data centers, paving the way to the large enterprise-scale intelligent systems we see today. Then in 2012, Arm revolutionized the embedded engineering industry when the first Arm-based *Raspberry Pi (Model B)* platform was launched, allowing hardware engineers and enthusiasts to experiment designs for the growing IoT ecosystem. This growth continues to accelerate to this day, and numerous organizations employ these computing devices into their commerce for production and scale [36]. Considering the enormous reach of these devices globally, it stands to reason that these computing devices form a critical part of security research and development, particularly firmware. The firmware of an IoT device contains various information that details the underlying functionality of the device and the surrounding IoT network that is attractive to malicious attackers. For this reason, the OWASP FSTM specifically chose the *Embedded Analyzer* (EMBA) [2][12][31], a comprehensive toolkit for automated firmware security analysis. This system intelligently extracts firmware binaries and performs static & dynamic analysis to uncover vulnerabilities and collects this data for reporting in security assessments. In this paper, we will demonstrate the OWASP FSTM and the *EMBA* to perform firmware analysis on a *Raspberry Pi* device operating an Arm Cortex-A processor. As a complement to our firmware analysis, *Tenable Nessus*, an enterprise-level vulnerability scanner, will be used to conduct network-based scans on our target device [11]. We will use *Tenable Nessus* as a vulnerability assessment platform to identify software flaws, misconfigurations, and compliance issues on an Arm-based IoT computer. The scan results produced by Nessus against a database of known vulnerabilities will demonstrate vulnerability information using established scoring systems to provide visibility into the external security of the IoT device [39]. These network-level findings will complement the firmware analysis performed on the ARM-based IoT device by *EMBA*, which will provide insight into the internal security of the device.

We will then explore how we can design a complete vulnerability lifecycle solution that repairs these intelligent systems dynamically and on demand. We aim to demonstrate an improved workflow for vulnerability management that is cost-effective, vendor-support free, does not require access to source code, and is entirely independent to create a legitimate lifecycle solution for these complex devices. Previously researched frameworks will be studied to refine patch management models that need improvement, thus bringing cohesion to the modern architecture that is required to keep ARM-based devices secure. This study will dive into legitimate CVE vulnerabilities that affect these devices and display how these real-world vulnerabilities play a role in authentication weaknesses within IoT systems. Furthermore, we aim to establish a professional, scalable lifecycle framework that can enable any enterprise to manage and mitigate security risks associated with ARM-based IoT devices.

II. Background

The Internet of Things is a proliferating ecosystem formed by 75 billion inter-connected devices over the Internet

[2]. Due to the vast networks that inter-connected IoT devices form, they are naturally a target for vulnerability researchers and attackers worldwide. Studies on IoT firmware security show that a significant portion of reported CVEs originated from weaknesses at the firmware layer, such as hard-coded credentials, weak authentication controls, outdated libraries, and insufficient input validation.[21] These weaknesses are not confined to a single category of devices but are commonly observed across consumers, industrial, and healthcare IoT systems. Since firmware is not regularly updated and typically lacks continuous security oversight, many of these vulnerabilities remain unpatched for extended periods. From an enterprise governance and risk perspective, firmware-level CVEs represent a serious concern, as compromised IoT devices can be exploited to gain unauthorized access, disrupt critical operations, or serve as entry points for broader attacks across interconnected environments. This highlights the increasing difficulty organizations face in maintaining visibility, enforcing security controls, and managing risk in IoT deployments scale [21]. According to the CVE published records, the number of reported vulnerabilities has continued to grow year after year, increasing from 25,059 in 2022 and 28,961 in 2023. This trend intensified in 2024 with 40,077 reported vulnerabilities, and by 2025 the total has reached 48,244. This increase indicates that securing and maintaining IoT systems is becoming more complex. In addition, more than 70,000 CVE IDs were reserved in 2025, showing that many vulnerabilities are known but not yet fully addressed [24]. A representative example is CVE-2024-41637, affecting ARM-based *Raspberry Pi* devices configured as wireless routers with RaspAP and default settings. To gain a foothold into the ARM-processor, the attacker's exploit path generally follows certain stages by exploiting an administrative failure in securing the default administrative credentials; a remote attacker then leverages a low-privileged web user (www-data) to escalate with password-less root privilege, writes to the *systemd* service file to modify the *restapi.service* with malicious control code, restarts the service to gain root access, bypassing normal authentication and authorization controls. CVE-2024-41637 received High severity rating and CVSS 3.1 score of 8.3, highlighted by the improper neutralization of special elements used in command injection, Common Weakness Enumeration (CWE)-77[46][47][49]. A device without any firmware-level authentication security would be vulnerable to this attack and potentially cause further damage by allowing modifications to the firmware. A similar case is CVE-2026-0715, which affects ARM-based industrial devices and allows access to the bootloader using device specific credentials that can be abused to disrupt device operation and cause temporary denial of service conditions, with a CVSS severity score of 7.0, highlighting ongoing weaknesses in credential and access handling ARM-based IoT environment [24].

To address and mitigate these cybersecurity threats, enterprise administrative efforts regularly update and distribute software "patches", or temporary repairs to the software components that are found to be vulnerable via software security testing. Due to the sheer volume, size, and complexity of enterprise devices or endpoints, in combination with the need to sustain business continuity and greatly minimize disruptions, these patches aren't normally deployed to the entire organization to reduce the impact that failed software updates, incompatible device architectures, and legacy components introduce to this aspect of device lifecycle management. Patch management is a security best practice, a common IT policy, industry standard, and legal compliance requirement for many enterprises that deploy IT infrastructure [17][27]. Particularly mobile devices, sensors, and other Internet of Things (IoT) devices, present an enormous challenge for organizations who operate heterogenous systems, such as industrial controls systems and manufacturing enterprises or those operating critical infrastructure [10]. Ensuring the security and integrity of firmware code is essential to defend against malicious attacks and unauthorized data access. Firmware modification attacks have plagued embedded devices, including those with integrated Arm circuitry for decades [38]. The arrangement and connection of these devices, their increasing complexity, and their global presence speak to the difficulty in meeting compliance and regulation requirements that govern the security and privacy of consumer & government information. Enterprise IT Administrators, namely the Chief Security Officers (CISO), are tasked with securing enterprise systems and endpoints. A leading study by S&P Global Market Intelligence in 2023 found that the main IoT security threat was unpatched software vulnerabilities, highlighting the stark difficulty in implementing successful patching campaigns. With the advent of IoT devices being integrated into traditional on-premises and cloud environments, current administrative efforts have trended towards securing these intelligent IoT systems employing a Zero-Trust model. Zero-Trust is a cybersecurity paradigm that requires organizations to assume breach and verify every access attempt regardless of its origin to protect against data breaches and enhance network visibility. Additionally, the proliferation of AI-enabled threats has galvanized more than three quarters of global organizations to actively transform their digital footprint by operationalizing Zero-Trust security. Patch management strategies delivered through cloud infrastructure and services are an aspect of this increasing trend, vital for responding to software vulnerabilities, particularly unknown zero-day vulnerabilities that present the highest risk to organizational assets [9][27][30][32][43][45][48].

Application Security Posture Management (ASPM) is an approach to assessing these risks and vulnerabilities. With many organizations' predominant use of applications, this security assessment tool helps provide support by mitigating risks of data breaches and potential data exfiltration. It is estimated that by 2026, over 40% of organizations which develop applications will utilize ASPM tools [34]. ASPMs provide continuous security visibility to enterprises, enabling constant insight into vulnerabilities and potential blind spots as opposed to frequent, manual security assessments. Additionally, ASPMs allow for integrating vulnerability management systems by aggregating security data from multiple sources into one console, thereby optimizing visibility into an organization's security posture. Despite these benefits, ASPMs face challenges in visibility when auditing the data obtained from IoT endpoints that supply critical network telemetry often because devices are difficult to identify, authenticate, and monitor consistently. Many of these devices operate with older software and hardware, rely heavily on Wi-Fi connectivity, and may lack proper authentication, which complicates asset discovery and limits effective auditing. These limitations become especially critical when zero-day vulnerabilities appear, as firmware and software updates are notoriously difficult to apply on IoT devices. Since immediate patching is often not feasible and

sometimes very latent, past and current security efforts have focused on isolating devices, network segmentation, and limiting exposure to reduce potential impacts. Considering the state of an ever-evolving modern threat landscape and AI-powered attacks, the challenge of securing infrastructure through establishing trust, visibility, and control over an overwhelming larger IoT ecosystem, securing machine identities is now the primary paradigm that humanity must be able to control. Our study will demonstrate a process that aims to support and address the problems that plague the initiatives set forth by enterprise administrators and security researchers.

The OWASP IoT Security Testing Guide (ISTG) was designed to outline the many challenging vulnerabilities associated with IoT devices. This guide provides structured methodology to assist in assessing security risks within firmware, hardware, network, and other features within IoT devices. With a strong focus on the authentication vulnerabilities of IoT devices in this guide, the ISTG is a strong framework for evaluating the potential risks and CVEs that may be found in an ARM-based device [15]. The OWASP Firmware Security Testing Methodology is a complementary extension of the OWASP ISTG, that involves a nine-stage approach to firmware security assessment. The process begins with information gathering and reconnaissance where all the technical documentation about the firmware is collected. A copy of the firmware binaries is then obtained for examination. The target firmware is then pre-processed for instance-specific implementation details. The next stage extracts the contents of the filesystem. Then the filesystem is statically analyzed for configuration files and software vulnerabilities. The firmware is then emulated in a virtual environment or sandbox. Dynamic analysis is then performed for application security testing. A runtime analysis is then performed, and the software bill of materials (SBOM) is rendered. In the final stage, we modify the written FSTM standard bypassing exploitation efforts, electing instead to generate binary patching information against the resultant vulnerabilities to provide on-demand patching, with a focus on autonomous application [10][16][21][22].

Automation provides many benefits to organizations, such as minimizing time consumption, manpower, and reducing human error. Automation has also become typical in security assessments, prompting the widespread usage of *Tenable Nessus*. This tool is one of the leading security assessment tools that searches for default configurations, exposed services, and any other potential authentication issues through network scanning and reports those vulnerabilities [11]. *Tenable Nessus* 10.11.x, the latest version as of January 2026, scans target endpoints remotely via IP ping sweeps and scan types such as Host Discovery, Vulnerability, Compliance, and other Expert Scans, which can be launched on a scheduled basis for enterprise endpoint security assessments. The Scan results show a summary dashboard indicating the issues discovered on targeted hosts revealing their compliance, vulnerability, and attack surface state. It also provides the necessary remediation information to resolve any severe security risks using the CVSS scoring system. *Tenable* also provides a complementing patch management solution, *Tenable Patch Management*, which ingests these reports and enhances security posture via automated distribution of patches to multiple cross-platform target endpoints, including ARM-based IoT devices. However, these *Tenable Nessus* clients require the installation of software agents on target devices to automate this distribution of update packages and unfortunately are subject to the same supply chain processes of vendor or developer distribution schedules. They also suffer from the added complexity of managing the proper installation and functioning of the product agents to provide optimal reliability of security posture data [17][18].

Moreover, there are inherent limitations when searching for security issues beyond the network via remote scanning. When assessing the internal security of ARM-based IoT devices, the lightweight nature of these devices minimizes visibility of the firmware binaries, since extensions to the CPU architecture are still a topic of research & development and are model architecture dependent [10][40]. To address this limitation, the OWASP FSTM promotes the use of a tool called the *Embedded Analyzer* (EMBA). *EMBA* is a powerful open-source tool and popular among security researchers due to its customizable framework and ease of use. The tool is a firmware security analyzer that tests the integrity of firmware filesystems, firmware files, and kernel configurations that interface with ARM-processors. It is composed of a small set of files primarily in the main program, *installer.sh* and *emba.sh,* which installs the program then executes shell commands to load the program modules to initiate the scan. Logging is supported, and the program consists of 35 modules that extend security assessment functionality. The tool is optimized for offline Linux-based firmware images. These binaries are the principal targets for exploitation and will inform the proceeding section [2][12].

Vendors for mobile and IoT operating systems typically distribute their product software and updates via binary files as compiled executables and do not disclose the source code for patched software. Other issues are that these vendor–supplied patches can introduce new vulnerabilities and at times the vendors supply patches that do not actually repair the target vulnerabilities, in what are known as "hidden patch gaps". Vendors also struggle with delivering patches in a timely schedule, which directly impacts the effectiveness of security teams in satisfying patch management objectives [10][43]. In the following section, we address potential solutions to these issues and their current pitfalls that require further studies.

III. Literature Survey & Related Work

Previous research on the study of IoT devices and their environments has focused on both security challenges and mitigation techniques. Recent work suggests utilizing features such as hardware-based security and even machine learning to minimize authentication issues in these devices [23]. A study conducted by Mefgouda *et al*. [20] proposed the use of PUFs (physical unclonable functions) as a robust security solution, noting that their randomness in manufacturing makes them unique enough to authenticate devices securely. These PUFs, though, are vulnerable to machine learning attacks and suffer from having hefty datasets, thus sparking the solution of the LPUF-AuthNet, a new, lightweight mechanism that merges neural networks with split learning to increase scalability and enhance security resistance [20].

This study also centers its focus on both cryptographic authentication and secure key exchange, which are frameworks that can be applied to ARM-based IoT devices.

Additionally, the LPUF-AuthNet utilizes autonomy in its authentication decision making process, using a verifier to decide if a device is legitimate using a Hamming distance comparison. In turn, rejection or approval can be decided without human intervention, providing a useful framework that we can base our autonomous patching framework on. Despite the advancements in authentication techniques highlighted in this study, research gaps exist addressing firmware integrity and security.

Binary patching has also been explored as a practical solution for securing ARM-based IoT devices when traditional update mechanisms are unavailable. A conceptual tool named *ARMPatch* presents a static binary patching framework that enables security fixes directly on compiled firmware through function replacement, code modification, and data updates. Addressing challenges for ARM and Thumb instructions, PC-relative addressing, and reference relocation after binary modification makes this patching difficult to operate, maintain, and risks firmware corruption. As a result, binary patching often struggles to scale effectively when security fixes require larger or structural changes to the firmware [19].

Based on these observations, the *ARMPatch* framework provides a concrete foundation for designing an improved binary patching tool. While its support for function replacement, instructional-level modification, data replacement, and dependency injection addresses a wide range of common vulnerability fixes, the limitations identified in its design highlight the opportunities for improvement. An improved binary patching framework can therefore be designed by extending the *ARMPatch* to better manage structural changes and improve patch robustness. For challenges associated with reference relocation and scalability, more resilient patching can be achieved. This enables security patches to be applied more reliably across diverse ARM-based IoT firmware [19].

Another tool we explored is *PATCHEKO*, a hybrid firmware analysis framework designed to detect known vulnerabilities and verify patch presence in closed-source mobile and IoT binaries. Static deep learning-based analysis can generate many false positives at scale, while dynamic analysis is more accurate but computationally expensive to apply broadly. Additionally, vulnerabilities detection alone does not address patch propagation challenges such as delayed updates and hidden patch gaps. As a result, these technologies cannot independently solve IoT security issues; only their integration enables reliable vulnerability identification and patch verification in real-world IoT firmware [10].

Together these tools form a roadmap towards building scalable, efficient, reliable, and necessary patching frameworks that can significantly improve the trustworthiness, authenticity, and security of IoT devices, and by extension an organization's security posture, supporting efforts to inform enterprise ASPMs with reliable and actionable threat intelligence and cybersecurity resilience.

IV. Experiment

This experiment consisted of a two-pronged approach to perform vulnerability assessment and firmware security scanning on a *Raspberry Pi 5*. The feature specifications for this device were a 64-bit quad-core ARM Cortex-A76 processor and 8GB of SDRAM, capable of communicating via IEEE 802.11ac Wireless Standard. Our goal in this experiment was to leverage the available vulnerability scanning tools to generate real vulnerability reporting to feed Application Security Posture Management systems. To achieve this goal, we first designed a simple wireless local area network (WLAN) using a separate local wireless router with data connectivity. We initialized the *Raspberry Pi* and bootstrapped the *Raspberry Pi OS* v5.1.2. Prior to setup, we launched a root shell and connected the device to the local WLAN that was created. We attached a host machine configured with *Tenable Nessus* 10.11.2 in the local subnet and in line of sight of the Raspberry Pi 5. We configured a Secure Shell (SSH) connection between the Nessus scanner and the *Raspberry Pi 5* device and performed a simple *ssh* connection targeting the *Raspberry Pi 5* device IP address to ensure there was connectivity. We then set up a Basic Network Scan Template [18] in *Tenable Nessus*, attached the required SSH credentials, and launched the scan with the associated default plugins that performed the vulnerability assessments. Once the scan ran to completion, we reviewed the results of the scan and generated an HTML report for future data analysis.

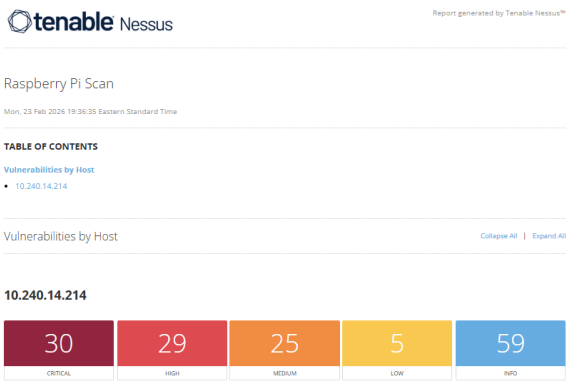
The next process in the experiment was to perform the OWASP Firmware Security Testing Methodology. The principal tool provided by OWASP to achieve the phases of the firmware security assessment prescribed by the OWASP FSTM is to utilize the *Embedded Analyzer* (EMBA). We first installed and configured the *EMBA* on a separate isolated and air-gapped Linux virtual machine. Within this environment, we began the first stage of the FSTM which is to perform reconnaissance on the *Raspberry Pi 5* and in configuring the *EMBA* with the necessary modules required for the scan. We completed the first stage of the FSTM by searching and identifying the location of the live firmware residency of the *Raspberry Pi 5*, specifically in this generation of the device the firmware is contained within the EEPROM microchip that exposes communication via the SPI interface in the Rasp Pi OS filesystem. The next stage of the FSTM involves firmware extraction. To accomplish this stage, we used the *flashrom* command targeting the *spidev10.0* interface to execute a live dump of the firmware into a raw binary file. Once the raw binary file was extracted, we cold transfer the firmware binary file to the secure Linux virtual machine using a USB disk to the directory storing the *EMBA* shell script. *EMBA* automates the Firmware Security Testing Methodology performing the firmware analysis, filesystem extraction filesystem extraction, firmware emulation, dynamic analysis, and runtime analysis [16], which we accomplished when we ran the *EMBA* against the firmware dump binary file. *EMBA* automatically aggregates its findings and generates an HTML file to report the information discovered.

V. Discussion of Results

Overall, in this experiment we successfully collected valuable security assessment information into an ARM-based IoT device. The complete *Tenable Nessus* scan results generated in the HTML file revealed a significant finding. *Raspberry Pi* devices, due to the period introduced by commercial storage and off-the-shelf to point sale, remain in a vulnerable state suffering software update neglect, which matches one of the underlying challenges in enterprise vulnerability management in delayed patch delivery. Because

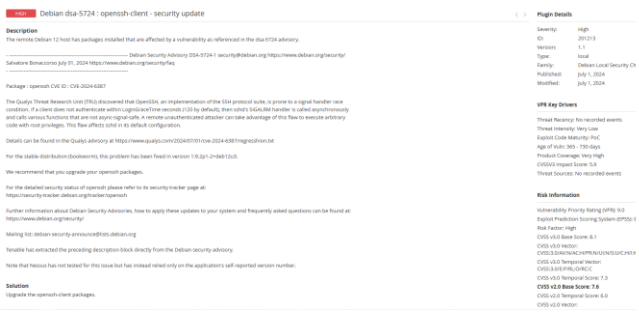
of the patch delay, the Nessus scan revealed 89 vulnerabilities with 30 critical vulnerabilities rated 10.0 in the CVSS 3.0 scale.

Figure 1: Nessus Basic Network Scan Complete Result



One serious vulnerability we discovered was CVE-2024-6387, that affects the OpenSSH client in its default configuration state. A remote unauthenticated attacker could exploit a race condition in the initial authentication stream and execute arbitrary code with root privileges. It received a High severity rating with a CVSS 3.0 base score of 8.1, formulated by CVSS:3.0/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H. To mitigate this threat, the recommended solution is upgrading the OpenSSH client packages, a solution that requires administrative effort.

Figure 2: CVE-2024-6387



The *EMBA* scan results were slightly underwhelming due to the added complexity required to overcome secure design features that recent firmware versions have implemented. *EMBA* successfully detected the *Raspberry Pi* firmware, but due to High Entropy in the raw binary file, most likely because of the firmware residing in EEPROM and *EMBA* recognizing a Real-Time Operating System (RTOS) or possible ARM-proprietary encryption, the embedded analyzer failed to detect the filesystem and successfully parse the binaries. This limitation can be mitigated by preprocessing the raw firmware binary with tools like Binwalk or Binvis to extract the filesystem for further analysis [31]. We used Binwalk to show the high entropy found in the initial sectors of the firmware file, matching the analysis performed by *EMBA*.

Figure 3: Binwalk firmware entropy analysis

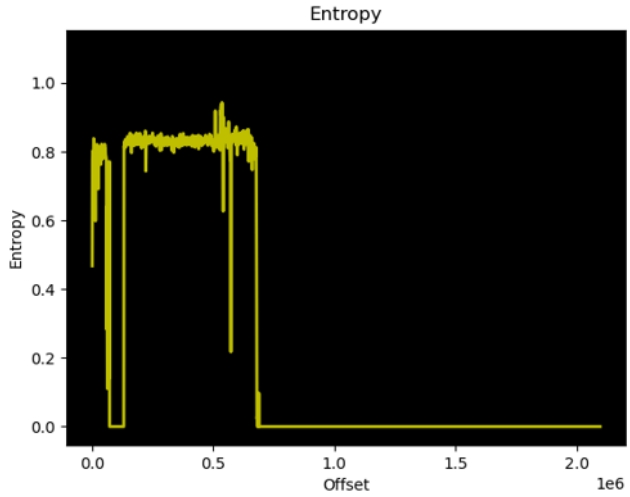


Figure 4: EMBA Binary firmware analysis, entropy.

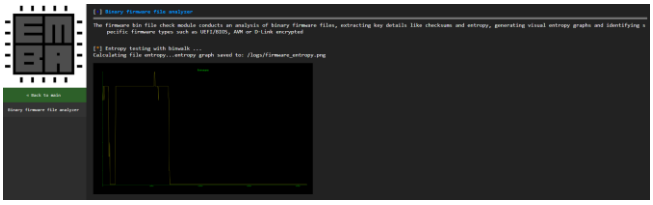


Figure 5: EMBA Binary firmware analysis, hash value.

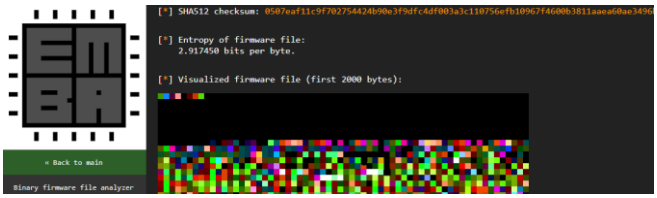


Figure 6: EMBA filesystem extraction

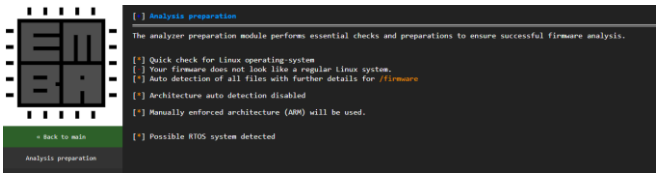
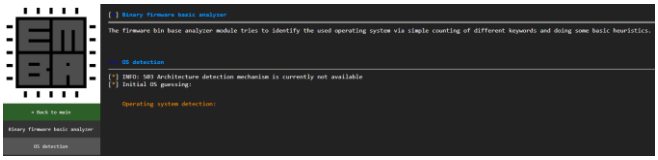


Figure 7: EMBA OS Detection

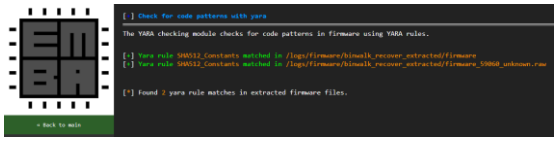


EMBA was able to discover the initial firmware directory and identify a few YARA rules that detect whether previous firmware extractions were performed.

Figure 8: EMBA filesystem tree

```
[*] 1 files and 3 directories detected.

==> Release/Version information

[-] No release/version information of target found

==> Filesystem information

[drwxr-xr-x        4096]  /logs/firmware
├── [drwxr-xr-x        4096]  binwalk_recover_extracted
│   ├── [lrwxrwxrwx           9]  firmware -> /firmware
│   ├── [-rw-r--r--       59044]  firmware_0_unknown.raw
│   ├── [-rw-r--r--          16]  firmware_59044_sha256.raw
│   └── [-rw-r--r--     2038092]  firmware_59060_unknown.raw
└── [drwxr-xr-x        4096]  unblob_extracted

3 directories, 4 files
```

Figure 9: EMBA YARA rule detection



VI. Conclusion

This study explored the security and vulnerability implications of network and firmware-level scans on ARM–based IoT devices, more specifically, on the *Raspberry Pi 5* platform. Utilizing *Tenable Nessus* to perform a remote vulnerability scan on a local network and an offline firmware security analysis using the *Embedded Analyzer* (EMBA) following the OWASP Firmware Security Testing Methodology (FSTM), we identified 90 vulnerabilities, 30 of which having a Critical rating CVSS 3.1 score of 10.0. The discovery of authentication-related vulnerabilities strongly demonstrates the broad attack-surface that ARM-based systems possess, and the gravity of not securing IoT devices out-of-box.

Our findings in this experiment reinforce that vulnerabilities in IoT devices are not limited to only one layer. Vulnerabilities such as outdated services, network visible misconfigurations, and other authentication issues with this layer show prevalence with firmware-level issues such as outdated embedded components and insecure configurations, among many other weaknesses. This highlights the importance of conducting deep scans on IoT devices that can identify major underlying issues within various layers, such as the weaknesses found within the Raspberry Pi device. Moreover, the results captured by the *EMBA* and *Tenable Nessus* scanner, when refined with adequate data pre-processing to address proprietary control mechanisms, can serve quality data streams to enterprise Application Security Posture Management systems by providing high-level aggregated data These procedures combined with future research investigating the development of automated patching systems to mitigate vulnerabilities on-demand can significantly improve the security posture of enterprise IoT systems and provide for an independent vulnerability lifecycle solution.

References

[1] M. Salayma, "Risk and threat mitigation techniques in internet of things (IoT) environments: a survey," Front. Internet Things, Sec. Security, Privacy and Authentication, vol. 2, Jan. 2024, Art. no. 1306018, doi: 10.3389/friot.2023.1306018

[2] BA Firmware Analyzer, "Leveraging Automated Firmware Analysis with the Open-Source Firmware Analyzer EMBA," Medium, Jul. 19, 2024. [Online]. Available: https://medium.com/@iugkhgf/leveraging-automated-firmware-analysis-with-the-open-source-firmware-analyzer-emba-46d30d587a87

[3] Statista, "Arm's market share and targets across key technology markets in 2019 and 2028 fiscal years," August 2020. https://www.statista.com/statistics/1132112/arm-market-share-targets/.

[4] "Developers' Newsletter No. 14," Acorn Computers Ltd. [Online]. Available: https://web.archive.org/web/20210521002744/http://chrisacorns.computinghistory.org.uk/docs/Acorn/DN/Acorn_DevNL14.pdf.

[5] "History of ARM: from Acorn to Apple," The Telegraph, 2011. [Online]. Available: https://web.archive.org/web/20180316092313/https://www.telegraph.co.uk/finance/newsbysector/epic/arm/8243162/History-of-ARM-from-Acorn-to-Apple.html.

[6] Marshall, Trevor; Tazelaar, Jane Morrill (February 1989). "Worth the RISC". BYTE. pp. 245–249. [Online] Available: https://archive.org/details/eu_BYTE-1989-02_OCR/page/n299/mode/2up

[7] "Arm A-profile A64 Instruction Set Architecture - Base Instructions," Arm Limited, Dec. 2025. [Online]. Available: https://developer.arm.com/documentation/ddi0602/2025-12/Base-Instructions?lang=en.

[8] "Smartphones," Arm Limited. [Online]. Available: https://www.arm.com/markets/consumer-technologies/smartphones.

[9] D. Kennedy, "Transition from isolation to exposure brings evolving threats to IoT and OT systems," 451 Alliance Blog, Jan. 14, 2026. [Online]. Available: https://blog.451alliance.com/transition-from-isolation-to-exposure-brings-evolving-threats-to-iot-and-ot-systems/.

[10] P. Sun, L. Garcia, G. Salles-Loustau and S. Zonouz, "Hybrid Firmware Analysis for Known Mobile and IoT Security Vulnerabilities," 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Valencia, Spain, 2020, pp. 373-384, doi: 10.1109/DSN48063.2020.00053

[11] Tenable, "Nessus Product Family," Tenable, 2023. https://www.tenable.com/products/nessus

[12] M. Messner and P. Eckmann, "EMBA - The firmware security analyzer," version 1.5.2, GitHub, Mar. 2025. [Online]. Available: https://github.com/e-m-b-a/emba

[13] ARM Lmtd. , "Arm Architecture Reference Manual, ARMv8, for ARMv8-A architecture profile," ARM Document DDI0487, [Online]. Available: https://developer.arm.com/documentation/ddi0487/latest/

[14] "OWASP Internet of Things," OWASP Foundation. [Online]. Available: https://owasp.org/www-project-internet-of-things/

[15] "OWASP IoT Security Testing Guide," OWASP Foundation. [Online]. Available: https://owasp.org/owasp-istg/

[16] Guzman, "OWASP Firmware Security Testing Methodology (FSTM)," version 1.0, OWASP Foundation, Nov. 2019. [Online]. Available: https://github.com/scriptingxss/owasp-fstm

[17] "What's New - Tenable Patch Management User Guide," Tenable, Inc., Jan. 22, 2026. [Online]. Available: https://docs.tenable.com/integrations/Tenable-Patch-Management/Content/whats-new.htm.

[18] "Scans - Nessus 10.11.x User Guide," Tenable, Inc. [Online]. Available: https://docs.tenable.com/nessus/Content/Scans.htm.

[19] M. Huang and C. Song, "ARMPatch: A Binary Patching Framework for ARM-based IoT Devices," in Journal of Web Engineering, vol. 20, no. 6, pp. 1829-1852, September 2021

[20] B. Mefgouda et al., "LPUF-AuthNet: A Lightweight PUF-Based IoT Authentication via Tandem Neural Networks and Split Learning," 2024 IEEE Globecom Workshops (GC Wkshps), Cape Town, South Africa, 2024, pp. 1-7, doi: 10.1109/GCWkshp64532.2024.11101578.

[21] B. Nancharaiah, J. G. A., G. S, F. A. Al-Naimi, J. Shubangi and D. Arumugam, "Establishment of Internet of Things (IoT) Firmware Security Flaws: Improving Cybersecurity with Dynamic and Static Code Analysis Tools," 2025 IEEE 5th International Conference on ICT in Business Industry & Government (ICTBIG), Indore, Madhya Pradesh, India, India, 2025, pp. 1-6, doi: 10.1109/ICTBIG68706.2025.11323540.

[22] S. Budiyanto, L. M. Silalahi, A. R. Hakim, A. Hamid and D. Hanafi, "Vulnerability Analysis on Internet of Things (IoT) Networks using Raspberry Pi and Open Web Application Security Project (OWASP)," 2024 FORTEI-International Conference on Electrical Engineering (FORTEI-ICEE), Badung, Indonesia, 2024, pp. 58-63, doi: 10.1109/FORTEI-ICEE64706.2024.10824490.

[23] Y. Xiao, Y. He, X. Zhang, Q. Wang, R. Xie, K. Sun, K. Xu, and Q. Li, From Hardware Fingerprint to Access Token: Enhancing the Authentication on IoT Devices, NDSS 2024, San Diego, CA, USA, Mar. 2024. [Online]. Available: https://arxiv.org/abs/2403.15271

[24] "Published CVE Records," CVE Program, 2025. [Online]. Available: https://www.cve.org/About/Metrics#PublishedCVERecords

[25] Roberts, E. "Pipelining." Stanford University, https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html.

[26] R. Lefferts, "CISO Insider: Issue 3 - Cloud-centric security: How leading CISOs close coverage gaps," Microsoft Security Insider, Apr. 26, 2023. [Online]. Available: https://www.microsoft.com/en-gb/security/security-insider/practical-cyber-defense/ciso-insider-issue-3.

[27] M. Souppaya and K. Scarfone, "Guide to Enterprise Patch Management Planning: Preventive Maintenance for Technology," National Institute of Standards and Technology, Gaithersburg, MD, NIST SP 800-40r4, Apr. 2022. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-40r4

[28] M. Gewida, "Microsoft best practices for managing IoT security concerns," Microsoft Security Blog, Apr. 25, 2022. [Online]. Available: https://www.microsoft.com/en-us/security/blog/2022/04/25/microsoft-best-practices-for-managing-iot-security-concerns/.

[29] "Introducing Azure Well-Architected Framework for Internet of Things (IoT)," Microsoft Tech Community - Azure Architecture Blog, Apr. 29, 2022. [Online]. Available: https://techcommunity.microsoft.com/blog/azurearchitectureblog/introducing-azure-well-architected-framework-for-internet-of-things-iot/3288539.

[30] G. Smith, "Design principles for unified edge device architecture," Microsoft Tech Community - Internet of Things Blog, Oct. 20, 2022. [Online]. Available: https://techcommunity.microsoft.com/blog/iotblog/design-principles-for-unified-edge-device-architecture/3638212.

[31] A. Guzman, "OWASP Firmware Security Testing Methodology," OWASP Foundation. [Online]. Available: https://scriptingxss.gitbook.io/firmware-security-testing-methodology/.

[32] "Architecture best practices for Azure IoT Hub," Microsoft Learn - Azure Well- Architected, Feb. 28, 2025. [Online]. Available: https://learn.microsoft.com/en-us/azure/well-architected/service-guides/azure-iot-hub?wt.mc_id=techcomm_wablog_blog_azuremktg

[33] "What is Application Security Posture Management (ASPM)? | CrowdStrike," Crowdstrike.com, 2019. https://www.crowdstrike.com/en-us/cybersecurity-101/application-security/application-security-posture-management-aspm/

[34] "Overview of Arm system architectures," Arm Limited, Learn the Architecture Series, Version 0100. [Online]. Available: https://developer.arm.com/documentation/110303/0100/Overview-of-Arm-system-architectures.

[35] "Platform firmware interfaces," in Learn the Architecture - Overview of Arm System Architectures, Arm Limited, Version 0100. [Online]. Available: https://developer.arm.com/documentation/110303/0100/Platform-firmware-interfaces?lang=en.

[36] Arm Editorial Team, "35 years of Arm innovation: The 35 Arm-powered products that redefined modern computing," Arm Newsroom, Nov. 26, 2025. [Online]. Available: https://newsroom.arm.com/blog/arm-35-years-technology-innovation.

[37] Roberts, E. "What Is RISC?" Stanford University. Available: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/whatis/index.html

[38] A. Cui, M. Costello, and S. J. Stolfo, "When firmware modifications attack: A case study of embedded exploitation," in Proc. Network and Distributed System Security Symp. (NDSS), San Diego, CA, USA, Feb. 2013.

[39] R. Awati, "What is the Nessus vulnerability scanning plataform?" TechTarget, Feb. 18, 2025. [Online]. Available: https://www.techtarget.com/searchnetworking/definition/Nessus

[40] S. Moore, "Enhanced Security through MTE," Arm Community, Oct. 20, 2020. [Online]. Available: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/enhanced-security-through-mte.

[41] Axcient, "ARM Processing and Its Impact on MSPs," Axcient, Nov. 21, 2024. [Online]. Available: https://axcient.com/blog/arm-processing-and-its-impact-on-msps/.

[42] emteria, "ARM vs. x86: The differences between the processor architectures," emteria, Oct. 17, 2024. [Online]. Available: https://emteria.com/blog/arm-vs-x86.

[43] Zscaler, "Why 81% of organizations plan to adopt zero trust by 2026," CIO, Apr. 15, 2025. [Online]. Available: https://www.cio.com/article/3962906/why-81-of-organizations-plan-to-adopt-zero-trust-by-2026.html

[44] Device Authority, "The State of IoT Identity Security in 2026: Why Machine Identity Is the New Perimeter," Device Authority, Jan. 29, 2026. [Online]. Available: https://deviceauthority.com/the-state-of-iot-identity-security-in-2026-why-machine-identity-is-the-new-perimeter/.

[45] V. Cooper, "Top Cybersecurity Trends and Predictions for 2026," Splashtop, Nov. 7, 2025. [Online]. Available: https://www.splashtop.com/blog/top-cybersecurity-trends-and-predictions-for-2026.

[46] A. Haymore, "CVE-2024-41637: Local privilege escalation in RaspAP," 0xzon.dev, Jul. 27, 2024. [Online]. Available: https://blog.0xzon.dev/2024-07-27-CVE-2024-41637/

[47] "CVE-2024-41637," National Vulnerability Database, NIST, Jul. 27, 2024. [Online]. Available: https://nvd.nist.gov/vuln/detail/cve-2024-41637.

[48] Splashtop, "Zero Trust Security: A Comprehensive Approach to Cybersecurity," Splashtop Blog, Jan. 10, 2024. [Online]. Available: https://www.splashtop.com/blog/zero-trust-security-a-comprehensive-approach-to-cybersecurity.

[49] MITRE, "CWE-77: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')," *Common Weakness Enumeration*, Dec. 11, 2025. [Online]. Available: https://cwe.mitre.org/data/definitions/77.html